



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

SC3000 - Artificial Intelligence

Lab 1

Lab Group	SCSC
Name	SOH CEK CONG
Matric Number	U2423500C
TA	RANDY ZAKYA SUCHRADY

Part 1 Finding a Shortest Path with An Energy Budget

Task 1:

We will use Uniform Cost Search to solve the shortest path problem. Starting from the start node S, we will expand the node with the least path cost $g(n)$. We repeat this until we find a path to the target node T. The steps are:

- Put start node S in a priority queue.
- Remove the node with the lowest path cost from the queue.
- Calculate cost to neighbour and add neighbour to priority queue.
- Add neighbours to a list of visited nodes so keep track of nodes that are visited and so no loop happens.
- Repeat until we reach target node T.

```
Task 1: -----
Shortest path: 1->1363->1358->1357->1356->1276->1273->1277->1269->1267->1268->1284->1283->1282->1255->1253->1260->1259->1249->1246->963->964->962->1002->952->1000->998->994->995->996->987->986->979->980->969->977->989->990->991->2369->2366->2340->2338->2339->2333->2334->2329->2029->2027->2019->2022->2000->1996->1997->1993->1992->1989->1984->2001->1900->1875->1874->1965->1963->1964->1923->1944->1945->1938->1937->1939->1935->1931->1934->1673->1675->1674->1837->1671->1828->1825->1817->1815->1634->1814->1813->1632->1631->1742->1741->1740->1739->1591->1689->1585->1584->1688->1579->1679->1677->104->5680->5418->5431->5425->5424->5422->5413->5412->5411->66->5392->5391->5388->5291->5278->5289->5290->5283->5284->5280->50
Shortest distance: 148648.63722140007.
Total energy cost: 294853.
```

We observe that the total energy cost for the shortest path exceeds the energy budget of 287932.

Task 2: Uninformed Search Algorithm

Uniform Search Algorithms include BFS, DFS, depth-limited search, iterative deepening search and uniform-cost search. Since BFS and DFS work only when edge distances are equal, we will use uniform-cost search for task 2.

The steps are:

- Put start node S in a priority queue.
- Remove the node with the lowest path cost from the queue.
- Calculate distance and energy cost to neighbours. If the energy cost exceeds the budget then skip this neighbour.
- Add the neighbour to the priority queue if a shorter path or a less energy path to this node is found.
- Repeat until we reach target node T.

```
Shortest path: 1->1363->1358->1357->1356->1276->1273->1277->1269->1267->1268->1284->1283->1282->1255->1253->1260->1259->1249->1246->963->964->962->1002->952->1000->998->994->995->996->987->988->979->980->969->977->989->990->991->2465->2466->2384->2382->2385->2379->2380->2445->2444->2405->2398->2395->2397->2142->2141->2125->2126->2082->2080->2071->1979->1975->1967->1966->1974->1973->1971->1970->1948->1937->1939->1935->1931->1934->1673->1675->1674->1837->1671->1828->1825->1817->1815->1634->1814->1813->1632->1631->1742->1741->1740->1739->1591->1689->1585->1584->1688->1579->1679->1677->104->5680->5418->5431->5425->5424->5422->5413->5412->5411->66->5392->5391->5388->5291->5278->5289->5290->5283->5284->5280->50
Shortest distance: 150335.55441905273.
Total energy cost: 259087.
```

We observe that the minimum distance is now longer than that of Task 1, as certain paths exceeded the budget.

Task 3: A* Search Algorithm

A*Star Search combines the idea of a greedy search with uniform-cost search by having an evaluation function: $f(n) = g(n) + h(n)$.

$f(n)$: total estimated cost for the path through a specific node.

$g(n)$: cost from start to the current node, which is known.

$h(n)$: estimated cost from the current node to the target node, which is estimated (the heuristics).

The $h(n)$ we use in this case will be the straight line distance from the specific node to the target node.

The steps are:

- Create a new dictionary named “H”, which contains the heuristics.
- We create another value in the key of the priority queue, which stores the value of $f(n)$. The priority queue is sorted by value of $f(n)$ and the node with the least $f(n)$ will be expanded first.
- The rest of the steps are similar to Task 2.

```
Shortest path: 1->1363->1358->1357->1356->1276->1273->1277->1269->1267->1268->1284->1283->1282->1255->1253->1260->125
9->1249->1246->963->964->962->1002->952->1000->998->994->995->996->987->988->979->980->969->977->989->990->991->2465-
>2466->2384->2382->2385->2379->2380->2445->2444->2405->2406->2398->2395->2397->2142->2141->2125->2126->2082->2080->20
71->1979->1975->1967->1966->1974->1973->1971->1970->1948->1937->1939->1935->1931->1934->1673->1675->1674->1837->1671-
>1828->1825->1817->1815->1634->1814->1813->1632->1631->1742->1741->1740->1739->1591->1689->1585->1584->1688->1579->16
79->1677->104->5680->5418->5431->5425->5424->5422->5413->5412->5411->66->5392->5391->5388->5291->5278->5289->5290->52
83->5284->5280->50
Shortest distance: 150335.55441905273.
Total energy cost: 259087.
```

A*Star Search is efficient but not always optimal. In this case, we got the optimal solution as our heuristic is admissible. We never overestimate the true distance cost to reach the goal state because we used straight line distance as our heuristic, which is always shorter than the actual street distance. Hence, the algorithms will not skip the best path.

Comparing Task 2 and Task 3:

With reference to the diagram below, we can see that without a heuristic (Task 2), we expanded 39040 nodes. With a heuristic (Task 3), the number of expanded nodes decreased to 3575, which is only 9% of what we got in Task 2.

```
Shortest path: 1->1363->1358->1357->1356->1276->1273->1277->1269->1267->1268->1284->1283->1282->1255->1
>989->990->991->2465->2466->2384->2382->2385->2379->2380->2445->2444->2405->2406->2398->2395->2397->214
>1931->1934->1673->1675->1674->1837->1671->1828->1825->1817->1815->1634->1814->1813->1632->1631->1742->
3->5412->5411->66->5392->5391->5388->5291->5278->5289->5290->5283->5284->5280->50
Shortest distance: 150335.55441905273.
Total energy cost: 259087.
Number of nodes expanded: 39040
Shortest path: 1->1363->1358->1357->1356->1276->1273->1277->1269->1267->1268->1284->1283->1282->1255->1
>989->990->991->2465->2466->2384->2382->2385->2379->2380->2445->2444->2405->2406->2398->2395->2397->214
>1931->1934->1673->1675->1674->1837->1671->1828->1825->1817->1815->1634->1814->1813->1632->1631->1742->
3->5412->5411->66->5392->5391->5388->5291->5278->5289->5290->5283->5284->5280->50
Shortest distance: 150335.55441905273.
Total energy cost: 259087.
Number of nodes expanded: 3575
```

Part 2 Solving MDP and Reinforcement Learning Problems Using a Grid World

Task 1:

Discount factor = 0.9

Assume step cost = -1 and getting into a goal state yield +10 without step cost (followed lecture slides).

Assuming stochastic transition model (Lecturer clarified on this),

$Q_{i+1}(s,a) = \sum_{s'} P(s'|s, a) [r(s,a,s') + \gamma V_{s'}]$ (probability * immediate reward + discounted future reward)

$$V_{i+1}(s) = \max_a [Q_{i+1}(s,a)]$$

$$\pi(s) = \operatorname{argmax}_a [Q(s,a)]$$

1.1 Value Iteration

Initialise all values in V table as 0.

For each iteration (until converges):

- For each (state, actions) calculate the new Q_{i+1} and update the Q table.
- Update the new V_{i+1} in the V table.

Update the policy after nth iteration by observing which action gives a higher Q value for each state S.

```
Part 2 Solving MDP and Reinforcement Learning Problems Using a Grid World ---
-----
Task 1: -----
V Table for value iteration:
[2.680720832249756, 4.567504317088077, 6.742925689841091, 9.27966101691769, 0]
[1.6761437997655182, 3.350249348138996, 5.052293846722705, 7.161016948949101,
9.27966101691769]
[0.25221770676842953, 0, 3.216648138615721, 0, 6.928482843534706]
[-0.8951775159765039, 0.042386088389384596, 1.5637974831443524, 2.711073004188
072, 4.651103501211867]
[-1.903277527075622, -0.970634040373709, 0.15685542757665683, 1.31424861953351
17, 2.7110730041880724]

Policy for value iteration:
['Right', 'Right', 'Right', 'Right', None]
['Right', 'Right', 'Right', 'Right', 'Up']
['Up', None, 'Up', None, 'Up']
['Up', 'Right', 'Up', 'Right', 'Up']
['Up', 'Right', 'Up', 'Right', 'Up']
Time taken: 0.0030 seconds
```

The V table is as such: (we assume no step cost when getting into goal state based on lecture slides)

2.6807	4.5675	6.7429	9.2797	0
1.6761	3.3502	5.0523	7.1610	9.2797
0.2522	0	3.2166	0	6.9285
-0.8951	0.0424	1.5638	2.7111	4.6511
-1.9032	-0.9706	0.1569	1.3142	2.7111

The policy will be as such:

Right	Right	Right	Right	
Right	Right	Right	Right	Up

Up		Up		Up
Up	Right	Up	Right	Up
Up	Right	Up	Right	Up

1.2 Policy Iteration

Initialise all actions in the policy table as randomly as shown below. Initialise all values in V table as 0.

```
#initialise policy
policy = [
    ["Up", "Down", "Left", "Right", "Up"],
    ["Up", "Down", "Left", "Right", "Up"],
    ["Up", "None", "Left", "None", "Left"],
    ["Up", "Down", "Left", "Right", "Up"],
    ["Up", "Down", "Left", "Right", "None"]
]
```

Repeat until policy is stable:

- Policy Evaluation:
 - Initialise a small value ω (e.g 0.001) and Δ as 0.
 - For each state, calculate the new V_{i+1} and update the V table. Calculate how much V_{i+1} changes as compared to V_i and update Δ if the change is larger than Δ .
 - Repeat Policy Evaluation until the V value converges ($\max \Delta < \omega$).
- Policy Improvement: For each (state,action) pair, calculate the Q(s,a) values and choose the action that gives the V(s) value. If the action is different, update the action in the policy table.

We stop the iteration if our policy doesn't change.

```
V Table for policy iteration:
[2.680721037035371, 4.567504335856348, 6.742925691841153, 9.279661016949152, 0]
[1.676144687645209, 3.350249416707186, 5.052293859689538, 7.161016949152543, 9.279661016949152]
[0.2522246031700365, 0, 3.216648267044473, 0, 6.9284828441504756]
[-0.8951456459987032, 0.042397572557972135, 1.5637991959932644, 2.7110740243447373, 4.651103637339966]
[-1.90317961404315, -0.9705955222721021, 0.15686455353093834, 1.3142527029881732, 2.7110740243447373]

Policy for policy iteration:
['Right', 'Right', 'Right', 'Right', 'None']
['Right', 'Right', 'Right', 'Right', 'Up']
['Up', 'None', 'Up', 'None', 'Up']
['Up', 'Right', 'Up', 'Right', 'Up']
['Up', 'Right', 'Up', 'Right', 'Up']
Time taken: 0.0040 seconds
```

The V table is as such: (we assume no step cost when getting into goal state based on lecture slides)

2.6807	4.5675	6.7429	9.2797	0
1.6761	3.3502	5.0523	7.1610	9.2797
0.2522	0	3.2166	0	6.9285
-0.8951	0.0424	1.5638	2.7111	4.6511
-1.9032	-0.9706	0.1569	1.3142	2.7111

The policy will be as such:

Right	Right	Right	Right	
Right	Right	Right	Right	Up
Up		Up		Up
Up	Right	Up	Right	Up
Up	Right	Up	Right	Up

1.3 Comparison between Value and Policy Iteration

Despite the 2 policies having different computation methods, both gave the same V table and same policy. This shows that both value iteration and policy iteration are equally good at deriving a good policy.

Task 2: Monte Carlo Control

Assume the transition is stochastic in this case, but the agent does not know the transition model. We also initialise a random policy as shown below.

```
policy = [ ["Up", "Up", "Up", "Up", "Up"],
           ["Up", "Up", "Up", "Up", "Up"],
           ["Up", None, "Up", None, "Up"],
           ["Up", "Up", "Up", "Up", "Up"],
           ["Up", "Up", "Up", "Up", None]
         ]
```

We repeat until convergence:

Generate a complete episode as such:

- Starting from the start state, each action we pick depends on policy.
- Each action has a small chance of deviating from the intended action.
- Repeat until the agent reaches the goal state.

Policy evaluation

For each (state,action) in episode:

- Find the first occurrence of (s,a).
- Calculate the total returns from the first occurrence to the goal state. Add the returns to a list.
- Update $Q(s,a)$ by averaging all the returns for that specific (s,a)

#Policy improvement

For each state in episode:

- Find the current optimal action, a^* , that gives the highest $Q(s,a)$.
- Update policy with the current optimal action, a^* , with a probability of $1-\epsilon$.
- Otherwise, with a probability of ϵ , update with a random action.

```

Task 2: -----
Policy for Monte Carlo:
['Right', 'Right', 'Right', 'Right', None]
['Right', 'Right', 'Right', 'Right', 'Up']
['Up', None, 'Up', None, 'Up']
['Up', 'Right', 'Up', 'Right', 'Left']
['Up', 'Right', 'Up', 'Right', 'Up']

```

The policy will be as such:

Right	Right	Right	Right	
Right	Right	Right	Right	Up
Up		Up		Up
Up	Right	Up	Right	Left
Up	Right	Up	Right	Up

2.1: Comparison between Policy of Task 1 and Task 2:

For reinforce learning using Monte Carlo, agents learn from actual episodes with stochastic transitions. As the transition has a bit of randomness, the state changes can be unpredictable. Hence, the policy that is obtained can be very different each time depending on our actual episodes.

Task 3: Q-Learning

Learning Rate = 0.1

$$Q_{new}(S_t, A_t) = Q_{old}(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q_{old}(S_{t+1}, a) - Q_{old}(S_t, A_t)]$$

$$\rightarrow Q_{new} = Q_{old} + \alpha [\text{immediate reward} + \text{discount future reward} - Q_{old}]$$

We do this:

- Initialise Q Table values as 0.

For each episode (loop until converges):

- For each step of episode:
 - Choose current optimal action, a^* , with a probability of $1-\epsilon$. Otherwise, with a probability of ϵ , choose with a random action.
 - After taking the intended action, observe the actual action. Actual action can be different from intended as the environment dynamic is stochastic.
 - Update new $Q_{new}(S_t, A_t)$ using the formula.
 - Update current state to new state.
 - If the agent reaches the goal, end the loop. Else, continue the process.

```

Task 3: -----
Policy for Q-Learning:
['Right', 'Right', 'Right', 'Right', None]
['Right', 'Right', 'Right', 'Right', 'Up']
['Up', None, 'Up', None, 'Up']
['Up', 'Right', 'Right', 'Right', 'Up']
['Up', 'Right', 'Right', 'Right', 'Up']

```

The policy will be as such:

Right	Right	Right	Right	
-------	-------	-------	-------	--

Right	Right	Right	Right	Up
Up		Up		Up
Up	Right	Right	Right	Up
Up	Right	Right	Right	Up

Compare Q-Learning policy with Monte Carlo Policy and Optimal Policy

All three policies are different. This is because there are ties involved in the process, so different actions can be chosen when their Q-values are the same. Due to the stochastic environment, Monte Carlo and Q-Learning tend to derive very random policies each time I run the test. Monte Carlo and Q-Learning also tend to converge at a later stage, so the policies will be very random when they haven't fully converged yet.

Efficiency and Convergence speed

- Time taken For Value Iteration: 0.0030 seconds
- Time taken For Policy Iteration: 0.0040 seconds
- Time taken For Monte Carlo: 0.0840 seconds
- Time taken For Q_Learning: 0.0060 seconds

Value & Policy Iteration is the fastest to converge as the transition probabilities are given and the V table and policy table can be computed easily.

Q-Learning is slower as it has to discover transitions by experience. However, as Q value changes after every transition, it learns relatively quickly as compared to Monte Carlo.

Monte Carlo is the slowest as it must wait until the end of an episode before making any updates. In larger grids, there are very infrequent updates and slow convergence.

Hence in terms of efficiency, Value&Policy iteration is the most efficient, followed by Q-Learning, then Monte Carlo.